

Arctic Wolf Software Developer Interview Prep Workbook

Executive Summary

This workbook is a comprehensive guide for preparing for the **Software Developer** interview at Arctic Wolf. We analyse the interview format, identify key algorithm/data-structure topics, and curate a broad set of practice problems (with answers and hints) tailored to this role. It includes a detailed 6-week study schedule, timed-practice strategies, mock-interview advice, and recommended learning resources. We also provide a salary estimate for this position in India. The content is organised into sections: interview rounds, DSA topics, curated problem lists, a study timeline (with a Gantt chart), daily study schedule, and preparation tips. All insights are grounded in recent interview reports and industry data [6†L175-L183] [7†L262-L270] [35†L60-L64] .

Interview Rounds at Arctic Wolf

Interviewees report that Arctic Wolf's software engineer process typically spans **4–6 rounds**. For example, one candidate described the process as: **(1)** recruiter screen, **(2)** high-level technical discussion, **(3)** two in-depth technical interviews, and **(4)** a behavioural interview [6†L175-L183] . Another report (for Bengaluru) outlines five rounds: **Round 1** – DSA coding (two medium LeetCode problems), **Round 2** – machine-coding assignment (small project to assess coding style), **Round 3** – paired programming (live React-based front-end implementation), **Round 4** – hiring manager discussion (leadership/fit), and **Round 5** – behavioural/cultural interview [7†L262-L270] . A high-level system-design question and standard behavioural questions often accompany the coding rounds [16†L201-L204] [7†L262-L270] . In early screening, expect one "medium/hard" LeetCode problem and one easy problem (with some high-level system or design discussion) [16†L201-L204] [7†L262-L270] .

Key DSA Topics and Patterns

Based on the job description and Arctic Wolf interview reports, the **most important data-structure and algorithm topics** are:

- **Arrays & Hashing:** Fundamental array manipulation, hash maps/sets for lookups. Common tasks include Two Sum, frequency counting, sorting-related problems [45†L203-L210] .
- **Strings & Sliding Window:** Classic string problems (substr/in-place operations). Sliding-window and two-pointer techniques solve "longest substring without repeating characters", minimum-window substring, etc [45†L207-L210] .
- **Trees & Graphs:** Binary tree traversals (DFS/BFS), tree recursion (path sums, LCA), graph connectivity (cycles, topological sort, BFS) [45†L209-L212] .
- **Sorting & Heaps:** Sorting algorithms, heap/priority-queue usage for "top K" and merge problems [45†L203-L210] .
- **Dynamic Programming:** Standard DP optimization (memoization) on subsequence/subset problems (knapsack, LIS, DP string problems) [45†L210-L212] .

- **Two-Pointers & Sliding-Window:** Overlaps with arrays/strings: two-pointer scans for sorted arrays (3Sum, container-with-most-water) and sliding-window for subarray problems (substring, max subarray) 【45†L207-L210】 .
- **Hashing & Maps:** Hash-based counting or caching (anagrams, frequency, mapping tasks) 【45†L203-L210】 .

Glassdoor reports highlight array/string problems (e.g. “*longest substring without repeating characters*”, “*next greater element*”) and a heap-based problem in coding rounds 【16†L293-L298】 【16†L239-L242】 . In practice, focus especially on **arrays, strings, hashing (maps/sets), two-pointer/sliding-window techniques, and tree traversals**, as these appear frequently and form the basis of many interview questions 【16†L293-L298】 【45†L203-L212】 .

Curated Practice Problems

We have assembled a wide range of practice problems (Easy/Medium/Hard) covering the topics above. Each entry includes a brief description, core idea, time complexity, a link to a practice platform (e.g. LeetCode or GeeksforGeeks), and a one-line hint. Use the problems below to guide your practice.

Arrays & Hashing

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
1	Two Sum	Given an array <code>nums</code> and a target, return indices of two numbers summing to target.	Use a hash map to store seen values and complements.	$O(n)$	LeetCode – Two Sum 【45†L227-L234】	Check complements in one pass to solve in $O(n)$ time.
2	Contains Duplicate	Check if any integer appears twice in an array <code>nums</code> .	Use a set to track elements and detect duplicates.	$O(n \log n)$ or $O(n)$ (hash set)	LeetCode – Contains Duplicate	A hash set insertion failure indicates a duplicate.
3	3Sum	Given an array, find unique triplets that sum to zero.	Sort array and use two-pointer scan for each element.	$O(n^2)$	LeetCode – 3Sum	Sort and fix one element, then two-sum on the remainder.
4	3Sum Closest	Find a triplet in array whose sum is closest to target.	Sort and two-pointers; track closest sum.	$O(n^2)$	LeetCode – 3Sum Closest	Similar to 3Sum but minimize absolute difference to target.

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
5	Container With Most Water	Given heights, find max area of water container (two lines).	Two-pointer from ends, move smaller inward.	$O(n)$	LeetCode – Container With Most Water	Move the pointer at the shorter line inward to find larger area.
6	Product of Array Except Self	Return array where each output[i] is product of all nums except nums[i].	Two-pass prefix/suffix product without division.	$O(n)$	LeetCode – Product of Array Except Self	Compute prefix products then multiply by suffix products.
7	Intersection of Two Arrays II	Return the intersection of two integer arrays (including duplicates).	Use hash maps or two-pointer after sorting.	$O(n + m)$	LeetCode – Intersection of Two Arrays II	Store counts in one array and scan the other.
8	Sort Colors (Dutch National Flag)	Sort array of 0s, 1s, 2s in-place.	Three-way partitioning (low, mid, high pointers).	$O(n)$	LeetCode – Sort Colors	Use three pointers (0,1,2) and swap to partition.
9	Merge Intervals	Given a list of intervals, merge all overlapping intervals.	Sort by start time, then merge overlapping ranges.	$O(n \log n)$	LeetCode – Merge Intervals	Sort intervals first; merge if current start $\leq$ last end.
10	Move Zeroes	Move all 0s in an array to the end, preserving order of nonzeros.	Two-pointer swap, or maintain a write index.	$O(n)$	LeetCode – Move Zeroes	Use one pass: swap non-zeros forward.
11	Find All Numbers Disappeared in an Array	Find numbers in [1..n] not present in <code>nums</code> .	Mark seen elements by sign or index.	$O(n)$	LeetCode – Find All Numbers Disappeared	Use index marking (negation) to track presence.
12	Longest Consecutive Sequence	Longest streak of consecutive integers in array.	Use a hash set, expand from each start of sequence.	$O(n)$	LeetCode – Longest Consecutive Sequence	Only start counting at sequence heads (num-1 not in set).

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
13	Subarray Sum Equals K	Count subarrays with sum = K.	Prefix-sum + hash map to count previous sums.	O(n)	LeetCode - Subarray Sum Equals K	Use prefix-sum and count occurrences of (currentSum - K).
14	4Sum	Given array, find unique quadruplets that sum to target.	Sort and two-pointer on inner two after fixing two.	$O(n^3)$	LeetCode - 4Sum	Extend 2-pointer technique from 3Sum by fixing two elements.
15	Next Permutation	Rearrange array into lexicographically next permutation.	Find pivot from right, swap, and reverse suffix.	O(n)	LeetCode - Next Permutation	Identify first descending pair from end, then swap with just larger, reverse rest.

Strings

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
16	Longest Substring Without Repeating Characters	Given a string, find the length of the longest substring with all distinct characters.	Sliding-window with a set or map to track last positions.	O(n)	LeetCode - Longest Substring Without Repeating Characters 【16†L295-L298】	Use two pointers and a map of character -> last seen index.
17	Valid Anagram	Check if one string is an anagram of another.	Count characters using an array or hash map.	O(n)	LeetCode - Valid Anagram	Character counts (or sort both strings) must match.
18	Group Anagrams	Group strings that are anagrams of each other.	Use sorted string or character count as key in hash map.	$O(n \cdot k \log k)$	LeetCode - Group Anagrams	Sort each string or use frequency tuple as map key.

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
19	Longest Palindromic Substring	Find the longest palindromic substring in a given string.	Expand around center or Manacher's algorithm.	$O(n^2)$ (expand)	LeetCode - Longest Palindromic Substring	Check all centers (char or between chars) and expand.
20	Palindrome Permutation	Determine if a string's characters can be rearranged into a palindrome.	Count odd occurrences (at most one odd allowed).	$O(n)$	LeetCode - Palindrome Permutation (via LeetCode 266)	At most one char with odd count for palindrome.
21	Min Window Substring	Smallest substring of <code>s</code> containing all chars of <code>t</code> .	Sliding window with two-pointer + hash counts.	$O(n)$	LeetCode - Minimum Window Substring	Expand right until all chars covered, then shrink left.
22	Implement strStr() (Needle in Haystack)	Return the index of the first occurrence of <code>needle</code> in <code>haystack</code> .	KMP algorithm or sliding window.	$O(n+m)$ (KMP)	LeetCode - Implement strStr()	Use two-pointer or KMP to avoid repeated scanning.
23	Zigzag Conversion	Write string in zigzag form on given rows then read line by line.	Simulate by appending chars to row strings in cycle.	$O(n)$	LeetCode - ZigZag Conversion	Keep an array of strings for each row and simulate the zigzag.
24	Valid Palindrome (two-pointer)	Check if string is palindrome (ignoring non-alphanumerics).	Two-pointer from ends, skip non-letters.	$O(n)$	LeetCode - Valid Palindrome	Use two indices moving inward, compare lowercase chars.
25	URLify (Replace Spaces)	Replace spaces in string with <code>%20</code> , given true length.	In-place from end, count spaces then shift.	$O(n)$	Cracking the Coding Interview Q1.3 (common)	Work from end: replace spaces by shifting characters.

Linked Lists

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
26	Reverse Linked List	Reverse a singly linked list (return new head).	Iterate with three pointers (prev, curr, next).	$O(n)$	LeetCode - Reverse Linked List	Change <code>next</code> of each node to point to previous.
27	Merge Two Sorted Lists	Merge two sorted linked lists into one sorted list.	Two-pointer merging, dummy head.	$O(n)$	LeetCode - Merge Two Sorted Lists	Similar to merging arrays: pick the smaller head each time.
28	Linked List Cycle	Detect if a linked list has a cycle.	Floyd's cycle-finding (slow & fast pointers).	$O(n)$	LeetCode - Linked List Cycle	Advance one pointer by 1, another by 2; check meeting.
29	Remove Nth Node From End of List	Remove the Nth node from end (one pass).	Two-pointer gap: advance fast N steps, then move both.	$O(n)$	LeetCode - Remove Nth Node From End	Use dummy head; advance fast by N+1, then move both to remove.
30	Add Two Numbers	Add two numbers represented as linked lists.	Iterate and sum digits with carry.	$O(\max(m,n))$	LeetCode - Add Two Numbers	Simulate grade-school addition with carry in list form.

Trees

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
31	Maximum Depth of Binary Tree	Find maximum depth (height) of a binary tree.	DFS (recursion) on both children, take max +1.	$O(n)$	LeetCode - Maximum Depth of Binary Tree	Recursively return $1 + \max(\text{leftDepth}, \text{rightDepth})$.

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
32	Symmetric Tree	Check if binary tree is mirror-symmetric.	Recursively compare left subtree and right subtree.	$O(n)$	LeetCode - Symmetric Tree	Use recursion or queue: compare mirror nodes.
33	Validate Binary Search Tree	Check if tree is a valid BST.	In-order traversal or range limits.	$O(n)$	LeetCode - Validate Binary Search Tree	Use min/max constraints or in-order sequence check.
34	Lowest Common Ancestor of BST	Find LCA of two nodes in a BST.	Leverage BST property: go left/right based on values.	$O(h)$	LeetCode - Lowest Common Ancestor of a BST	If both values < node, go left; if >, go right; else current is LCA.
35	Binary Tree Level Order Traversal	Return nodes level by level (breadth-first).	BFS with queue, track levels.	$O(n)$	LeetCode - Binary Tree Level Order Traversal	Use a queue and iterate level by level.
36	Path Sum	Check if a root-to-leaf path sums to target value.	DFS tracking current sum.	$O(n)$	LeetCode - Path Sum	Subtract node value from sum as you go down.
37	Serialize and Deserialize Binary Tree	Convert binary tree to string and back.	Use preorder traversal with placeholders (e.g. "#").	$O(n)$	LeetCode - Serialize and Deserialize Binary Tree	Use a queue/recursion with a null marker.
38	Balanced Binary Tree	Check if a binary tree is height-balanced (AVL property).	Recursively check heights, return -1 if unbalanced.	$O(n)$	LeetCode - Balanced Binary Tree	Return height and a bool; bail out if any subtree differs >1.
39	Convert Sorted Array to BST	Given a sorted array, convert to height-balanced BST.	Recursively pick mid as root.	$O(n)$	LeetCode - Convert Sorted Array to BST	Choose middle element as root for balanced BST.

Graphs

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
40	Number of Islands	Count islands (connected regions of 1s) in 2D grid.	BFS/DFS on grid, mark visited.	$O(m \cdot n)$	LeetCode – Number of Islands	Run DFS/ BFS from each unvisited '1', mark entire island.
41	Clone Graph	Deep copy an undirected graph.	DFS/BFS with a hash map for cloned nodes.	$O(V+E)$	LeetCode – Clone Graph	Use a map from original to cloned node during DFS.
42	Course Schedule (Topological Sort)	Given prerequisites, find if can finish all courses (no cycle).	Use DFS cycle detection or Kahn's algorithm.	$O(V+E)$	LeetCode – Course Schedule	Perform topological sort; if cycle detected, return false.
43	Word Ladder	Transform beginWord to endWord changing 1 char each step.	BFS on words graph (each word edge if one-letter diff).	$O(N \cdot 26 \cdot L)$	LeetCode – Word Ladder	BFS from begin, generate all 1-letter variations at each step.
44	Number of Connected Components in Graph	Count connected components in an undirected graph.	DFS/BFS on each unvisited node.	$O(V+E)$	LeetCode – Number of Connected Components	Run DFS/ BFS from each unvisited node, count increments.
45	Pacific Atlantic Water Flow	Matrix with heights, find cells water can flow to both oceans.	Reverse BFS/DFS from oceans, intersect reachability.	$O(m \cdot n)$	LeetCode – Pacific Atlantic Water Flow	Start from edges and find all reachable cells.

Heaps & Sorting

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
46	Kth Largest Element in an Array	Find the k-th largest element.	Use a min-heap of size k, or Quickselect.	$O(n \log k)$ / $O(n)$ expected	LeetCode – Kth Largest Element	Maintain min-heap of size k; top of heap is answer.
47	Merge k Sorted Lists	Merge k sorted linked lists into one sorted list.	Use a min-heap to select smallest head among lists.	$O(N \log k)$	LeetCode – Merge k Sorted Lists	Push all list heads into heap; pop lowest, push next.
48	Top K Frequent Elements	Return k most frequent elements in array.	Use hash map for counts + min-heap of size k.	$O(n \log k)$	LeetCode – Top K Frequent Elements	Count frequencies then heap-select top k.
49	Meeting Rooms II	Given intervals of meeting times, find minimum number of conference rooms needed.	Sort by start time; use a min-heap of end times.	$O(n \log n)$	LeetCode – Meeting Rooms II	If next meeting starts after earliest end, reuse room; else allocate new.
50	Sort Characters By Frequency	Sort characters in string by decreasing frequency.	Count chars with hash map, then use max-heap.	$O(n \log n)$	LeetCode – Sort Characters By Frequency	Count and push (freq, char) into max-heap or bucket sort.

Dynamic Programming (DP)

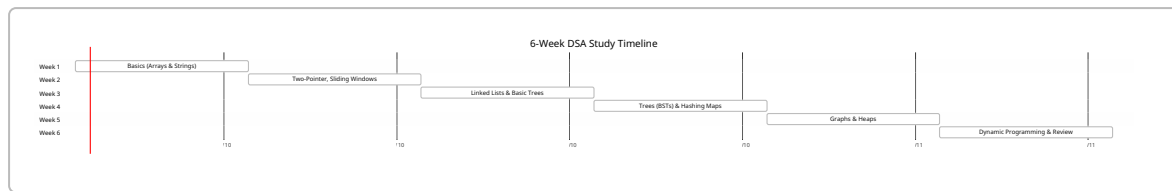
#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
51	Fibonacci Number (Classic)	Compute Nth Fibonacci (naively exponential).	Use DP/ memoization or iterative bottom-up.	$O(n)$	LeetCode – Fibonacci Number	Use an array or two variables to build up.

#	Title	Description	Key Idea	Time Complexity	Practice Link & Platform	Hint
52	Climbing Stairs	Count ways to climb n steps (1 or 2 steps at a time).	DP: $\text{ways}[n] = \text{ways}[n-1] + \text{ways}[n-2]$.	$O(n)$	LeetCode - Climbing Stairs	A classic Fibonacci-like DP recurrence.
53	House Robber	Max sum of non-adjacent elements in array.	DP: choose or skip each house.	$O(n)$	LeetCode - House Robber	DP: <code>dp[i] = max(dp[i-1], dp[i-2] + nums[i])</code> .
54	Longest Increasing Subsequence (LIS)	Length of longest strictly increasing subsequence.	DP or patience sorting ($O(n \log n)$).	$O(n^2)$ (DP)	LeetCode - Longest Increasing Subsequence	Standard DP: <code>dp[i] = 1 + max(dp[j])</code> for all $j < i$ if $\text{nums}[j] < \text{nums}[i]$.
55	Coin Change	Minimum coins to make up a target amount.	DP 1D: for each coin, update $\text{dp}[]$; or BFS.	$O(\text{amount} * \# \text{coins})$	LeetCode - Coin Change	Build dp array up from 0 to target, use each coin.
56	Edit Distance (Levenshtein)	Minimum edits (insert/delete/replace) to convert word1 to word2.	2D DP: <code>dp[i][j]</code> based on neighbors.	$O(m \cdot n)$	LeetCode - Edit Distance	<code>dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + cost</code> .
57	Word Break	Determine if string can be segmented into dict words.	DP with a boolean array, or DFS+memo.	$O(n \cdot m \cdot k)$	LeetCode - Word Break	Use $\text{dp}[i] = \text{true}$ if $\text{dp}[j]$ is true and $s[j:i]$ in dict.

Each table above provides a strong starting set of problems. For a full **60–100 practice list**, expand each category with similar examples (e.g. more array problems like “*median of two sorted arrays*”, graph problems like “*num of islands II*”, DP like “*knapsack*”, etc.), ensuring a mix of difficulties. Always review the problem post-solution to understand alternative approaches and edge cases.

6-Week Study Plan

We recommend a **6-week** structured plan, dedicating most weekdays to learning and practice, with lighter weekends for review. Below is a recommended schedule. Adjust topics based on your strengths/weaknesses. Each day mix learning concepts with 1–2 timed practice problems, and periodically revisit earlier topics.



Week	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
1	Arrays: Learn two-pointer and hash patterns (e.g. Two Sum, 3Sum); solve 1 easy array problem (timed).	Strings: Sliding-window on substrings (Long. Substr etc.); solve 1 string problem.	Arrays: Sorting-based array problems (Merge Intervals, Container water); solve 1 medium array problem.	Strings: Hash/count in strings (Valid anagram, group anagrams); solve 1 string problem.	Mixed: Quick review (flashcards) of Week1 topics, solve 1 easy/medium from weak area.	Practice Test: Do two timed problems covering arrays/strings under 45m (simulate interview conditions).	Rest & Review: Review mistakes, revise tricky points from Week1.
2	Two Pointers: Focus on sliding-window techniques for arrays (max subarray, min window); solve 1 medium.	Two Pointers: More array (Remove Duplicates, 4Sum); solve 1 medium/timed.	Sliding Window: Advanced string window problems (Minimum Window Substring); solve 1 hard sliding-window.	Hashing: Practice hash map problems (Subarray Sum = K); solve 1 prefix-sum problem.	Mixed: Review Week2 topics; solve 1 mixed problem (easy).	Practice Test: Timed set of 2 problems (arrays/strings).	Rest & Review: Go over errors, re-solve tough problems from Week2.
3	Linked List: Learn pointers (reverse list, detect cycle); solve 1 easy.	Linked List: Continue (merge lists, add numbers); solve 1 medium.	Trees: Basic tree DFS (max depth, symmetric); solve 1 easy.	Trees: In-order and search tree (validate BST, LCA); solve 1 medium.	Mixed: Quick review of lists/trees; solve 1 practice interview question mixing them.	Practice Test: Timed pair: one list + one tree problem.	Rest & Review: Revise list/tree patterns and tricky cases.

Week	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
4	Trees: Level order, path sums; solve 1 medium (e.g. level-order).	Hashing/ Maps: Complex data structures (serialize tree, LCA); solve 1 medium.	Heaps: Learn priority-queue problems (merge k lists, kth largest); solve 1 medium.	Heaps: More heap problems (top-k frequent); solve 1 medium.	Mixed: Review Week4 topics; attempt 1 problem from each area quickly.	Practice Test: Timed two problems: one tree, one heap.	Rest & Review: Deep dive on any unresolved heap or tree question.
5	Graphs: BFS/DFS fundamentals (number of islands); solve 1 medium.	Graphs: Topological sort / cycle detect (Course Schedule); solve 1 medium.	Graphs: Word ladder / connectivity; solve 1 hard.	Dynamic Programming: Intro (climbing stairs, Fibonacci); solve 1 easy.	DP: 1D DP problems (House Robber, Min path sum); solve 1 medium.	Practice Test: Timed two problems: one graph, one DP.	Rest & Review: Clarify graph search vs DP strategies; re-solve mistakes.
6	DP: 2D DP (Edit Distance, coin change); solve 1 hard.	DP: Review and additional patterns (LIS, knapsack); solve 1 medium.	Mixed Practice: Solve one problem each of two weak areas (45m total).	Mock Interview: Ask a peer or use a platform to do a full mock (including coding + design).	Review & Polish: Revisit any remaining weak topics (review notes, flashcards).	Practice Test: Final timed practice (2 problems of your choice).	Rest & Ready: Light review; prepare mental strategy and relax before interview.

- **Daily Routine:** On weekdays, spend 2–3 hours on concepts + practice. Attempt at least one problem per topic **with a timer (45–60 minutes)** to simulate the coding round. Immediately analyze errors after each problem.
- **Timed Practice Strategy:** Regularly simulate interview conditions by coding problems on paper or a blank editor within a fixed time. Focus on writing clean code and explaining your thought process aloud (even if alone) to build communication skills. Use platforms' "interview" or "contest" modes to enforce timing.
- **Mock Interviews:** In Weeks 5–6, schedule 2–3 mock sessions with peers or through services (Pramp, Interviewing.io, colleagues). Include both algorithm problems and design/cultural questions. Treat these like real interviews: introduce yourself, solve problems live, and seek feedback on solutions and communication.

Recommended Resources

- **Books:** *Cracking the Coding Interview* by McDowell; *Elements of Programming Interviews* by Aziz/ Lee/Prakash; *Competitive Programming* by Halim (for practice). These cover many of the above problems and patterns.

- **Online Courses & Videos:** MIT/Stanford OpenCourseWare (e.g. *Intro to Algorithms*), Coursera's *Algorithmic Toolbox*, free video playlists (e.g. WilliamFiset's Data Structures, Tushar Roy's algorithms).
- **Practice Platforms:** LeetCode (use *Discuss* and *Explore*), GeeksforGeeks (great for topic primers), HackerRank, Codeforces (for challenge practice). Use company interview lists on Glassdoor or LeetCode to find related problems.
- **Others:** Regularly review *system design* basics (since one round may involve high-level design). Brush up on Arctic Wolf's domain (security analytics, cloud) to handle any role-specific questions.

Estimated Salary (India)

For a Software Developer role at Arctic Wolf in India (e.g. Bengaluru), recent Glassdoor data suggests the range is roughly **₹27–30 LPA** [35†L70-L74] for mid-level developer positions. More broadly, Arctic Wolf 'Developer' roles span about **₹23–40 LPA** [35†L60-L64] , and *Senior Developer* roles go higher (up to ~₹50L). Depending on your experience, a reasonable expected salary to quote would be in the **25–35 LPA** range for a mid-level developer in Bangalore. (Always consider bonuses, stock options, and negotiate based on your skill fit.)

Sources: Interview reports and question lists (Glassdoor) [6†L175-L183] [7†L262-L270] [16†L239-L242] ; algorithmic topic summaries [45†L203-L212] ; Glassdoor salaries [35†L60-L64] [35†L70-L74] .
